

11/9/26

CS 340

Lecture 18

Ambiguity in CFGs

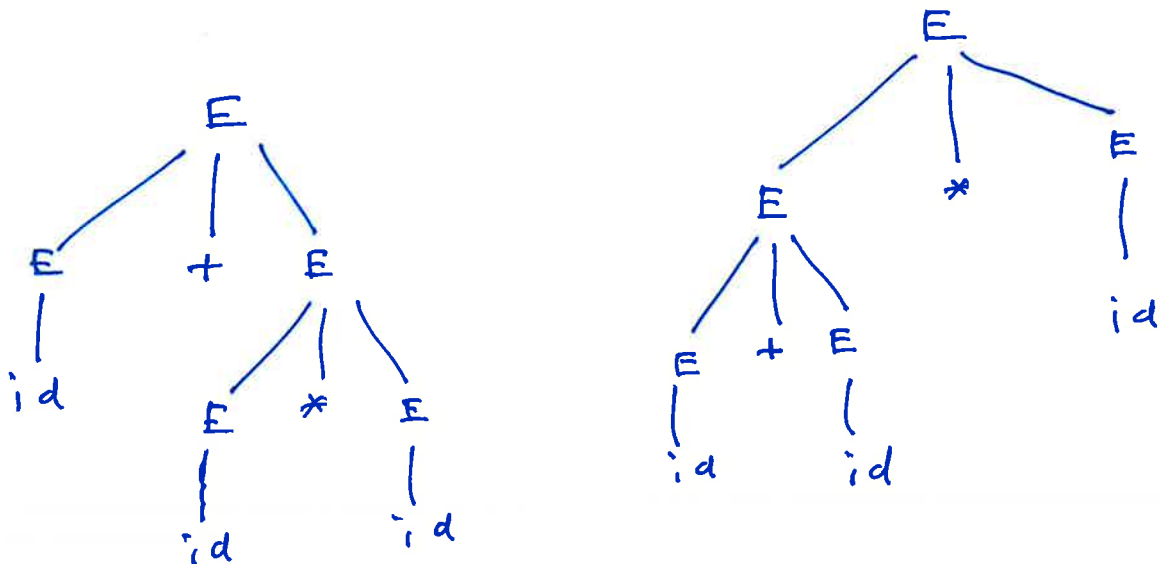
Ambiguity

A CFG G is said to be ambiguous if for some $w \in L(G)$ it has two distinct parse trees.

This is equivalent to saying that w has two distinct leftmost (rightmost) derivations.

Example:

string $id + id * id$ has two different parse trees in our grammar for arithmetic expressions.



Both parse trees here yield $id + id * id$

Parse tree for w is often used to give semantics to w . If there are more than one parse tree for w then its meaning becomes ambiguous. (This can happen in natural languages also)

~~the~~

In the grammar for arithmetic expressions, parse tree specifies order of ~~evaluation~~ evaluation of various operators. We evaluate an expression bottom up, starting at leaves and going all the way up to the root of the parse tree. This ensures that ~~as~~ if we apply the operator at any internal node in the tree then the operands have already been evaluated.

For our expression, first parse tree has the meaning $id + (id * id)$ and the second parse tree corresponds to $(id + id) * id$.

In our grammar, we can derive both the above expressions, with explicit parentheses. However, if we omit ~~parentheses~~ parentheses, then $id + id * id$ does not have a ^{unique} meaning. Clearly, it is desirable to have only one parse tree for ^{these} expressions also so that every expression denotes a well defined value.

In practice this is done by introducing ^a precedence ~~relation~~ relation on operators and specifying left/right associativity for operators at the ~~same~~ same precedence level.

We show how to incorporate this method in our grammar to make it unambiguous.

Suppose in an expression $+$, $*$ and $(\dots)$ appear at same level. Then $(\dots)$ is evaluated first, after that $*$ is evaluated and $+$ is evaluated in the end. So, we keep $(\dots)$ close to leaf and $+$ close to the root.

Following grammar does this by introducing two new categories factor (F) and term (T), apart from E .

$$F \rightarrow \text{id} \mid (E)$$

$$T \rightarrow T * F \mid F$$

$$E \rightarrow E + T \mid T$$

Grammar G'

In this grammar $\text{id} + \text{id} * \text{id}$ has only one ^{leftmost} derivation may be seen as follows. First rule must be of the form ~~$E \rightarrow E + T$~~ $E \rightarrow T$ because $E \rightarrow T$ can generate $'+'$ only inside a parenthesis.

$$E \Rightarrow E + T \Rightarrow \text{id} + T$$

second rule can only be $E \rightarrow T$ because $E \rightarrow E + T$ generates two occurrences of $'+'$. Similarly third rule can be $T \rightarrow F$ only (otherwise $'*'$ appears before $'+'$)

$$T + T \Rightarrow F + T \Rightarrow \text{id} + T$$

The next rule can only be $T \rightarrow T * F$ (because $T \rightarrow F$ can't generate $'*'$ without parentheses)

$$E \Rightarrow_* id + T \Rightarrow id + T * F$$

From T we can apply only $T \rightarrow F$ (otherwise we get extra '*')

$$E \Rightarrow_* id + \cancel{T} * F \Rightarrow id + F * F$$

Now from both F , we can apply only $F \rightarrow id$ because applying $F \rightarrow (E)$, would give rise to '(', ')' in the final string.

$$E \Rightarrow_* id + F * F \Rightarrow id + id * F \Rightarrow id + id * id.$$

To show that G' is unambiguous would be tedious, but for each $w \in L(G')$, we can use argument similar to above to show that there is only one leftmost derivation $E \xRightarrow_{lm}^* w$.

Exercise: write grammar G'' with the same language as $L(G)$ or $L(G')$ but '+' , '*' ^{should} associate from the right.

To prove $L(G) = L(G')$ is also a bit technical.

It is easy to show that $L(G') \subseteq L(G)$.

Exercise

show that if $F \Rightarrow_{G'}^* u$, $T \Rightarrow_{G'}^* v$, $E \Rightarrow_{G'}^* w$,

where $u, v, w \in \{id, +, *, (,)\}^*$ then

$E \Rightarrow_G^* u$, $E \Rightarrow_G^* v$ and $E \Rightarrow_G^* w$.

[use induction on w]

For the other direction, let $w \in L(G)$. If w does not have a unique parse tree, then consider the parse tree of w in G , which corresponds to usual operator precedence and associativity rules (same as in G'). Argue that this parse tree corresponds to a parse tree in G' .

Let us consider another example of an unambiguous grammar. It is grammar of arithmetic expression written in prefix form.

Example:

Consider grammar G

$$E \rightarrow +EE \mid *EE \mid -EE \mid x \mid y$$

give leftmost derivation for $+*-xyxy$

$$E \Rightarrow +EE \Rightarrow +*EEE \Rightarrow +*-EEEE$$

$\Downarrow$

$$+*-xyzE \leftarrow +*-xyEE \leftarrow +*-xEEE$$

$\Downarrow$

$$+*-xyxy$$

Can you prove that this grammar is unambiguous.

We can do this ~~or follows~~ by showing that there is a unique leftmost derivation for every $w \in L(G)$.

~~Ass~~ Suppose $E \Rightarrow^* w_1 E \alpha$ $w_1 \in T^*$
in the leftmost derivation of w

~~for this to~~

$$(\Rightarrow) w = w_1 w_2$$

there is a unique production rule applicable depending on the first symbol of w_2 . $\square$

It would be nice if we had a mechanical way to convert any CFG to an equivalent Unambiguous CFG.

This however, is not possible for two ~~is~~ different reasons.

1. Some CFLs are inherently ambiguous. That is every CFG for them is ambiguous.

Consider $L = \{a^i b^j c^k \mid i=j \vee j=k\}$
 $i, j, k \geq 1$

L may be generated by following simple CFG

$$S \rightarrow S_1 c \mid a S_2$$

$$S_1 \rightarrow S_1 c \mid D$$

$$D \rightarrow ab \mid a D b$$

$$S_2 \rightarrow a S_2 \mid E$$

$$E \rightarrow bc \mid b E c$$

Words $a^i b^i c^i$, for $i \geq 1$, have two different leftmost derivations. For example,

$$S \Rightarrow a S_2 \Rightarrow a E \Rightarrow abc$$

$$S \Rightarrow S_1 c \Rightarrow D c \Rightarrow abc$$

are two distinct leftmost derivations of 'abc'

This only proves that above grammar for L is ambiguous not that L is inherently ambiguous. Proof for inherent ambiguity of L ^{would be} more involved.

* Another example ~~is~~ in L_1 defined as

$$L_1 = \{ a^i b^i c^j d^j \mid i, j \geq 1 \} \\ \cup \{ a^i b^j c^j d^i \mid i, j \geq 1 \}$$

Exercise: show that L_1 is a CFL.

L_1 is known to be inherently ambiguous CFL.

2. There is no algorithm to decide ambiguity of a CFG. More precisely, there is no algorithm ~~is~~ with following input-output behavior

Input : a CFG G
Output : $\begin{cases} \text{yes} & \text{if } G \text{ is ambiguous} \\ \text{no} & \text{if } G \text{ is } \cancel{\text{ambiguous}} \text{ unambiguous} \end{cases}$

I would like to ^{cover} proofs of both 1 and 2, if we have sufficient time in the course.